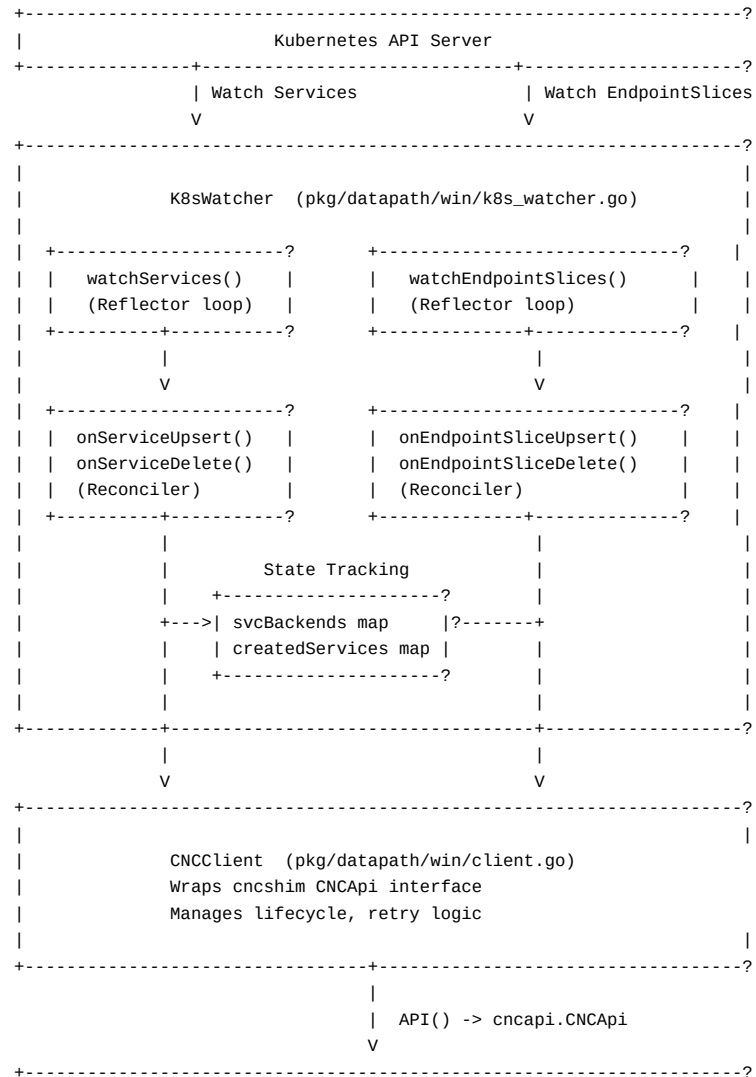
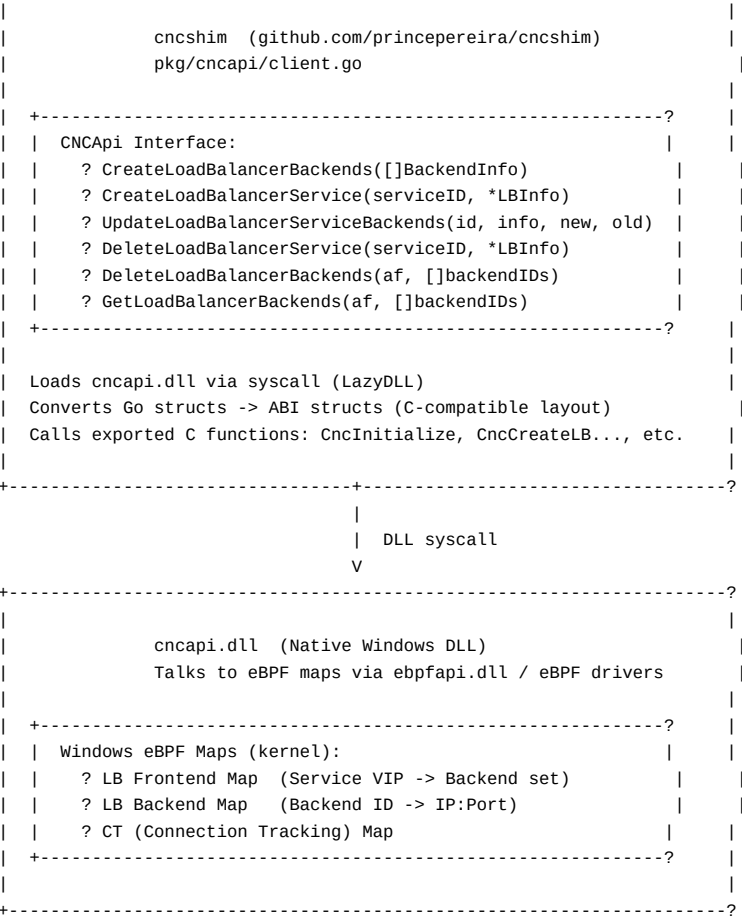


Cilium Windows Datapath Architecture

Overview

The Cilium Windows agent uses a layered architecture to watch Kubernetes resources and program Windows eBPF maps via the CNC (Container Networking Compute) API.





Component Details

1. Hive / Dependency Injection (daemon/cmd/windows_stubs.go)

The agent is bootstrapped using Cilium's Hive framework for dependency injection. Two cells are registered:

Cell	Module ID	Purpose
`winDatapath.Cell`	`datapath-win`	Provides `CNCCClient` (cncshim connection)
`winDatapath.K8sWatcherCell`	`k8s-watcher-win`	Watches K8s, programs LB rules

Hive manages the lifecycle: Start -> Run -> Stop.

2. CNCClient (pkg/datapath/win/client.go)

Role: Manages the connection to cncshim (cncapi.dll).

- On Start(): calls cncapi.New() which loads cncapi.dll and calls CncInitialize
- If initialization fails (DLL not ready), retries every 5 seconds in background
- Exposes API() method returning cncapi.CNCApi interface
- Other components check API() != nil before making calls

3. K8sWatcher (pkg/datapath/win/k8s_watcher.go)

Role: The core reconciler. Watches K8s resources and translates them into CNC API calls.

Watcher / Reflector Pattern

Two goroutines run Watch loops against the K8s API server:

Goroutine	Resource Watched	Event Handlers
<code>`watchServices()`</code>	<code>`v1.Service`</code>	<code>`onServiceUpsert`, `onServiceDelete`</code>
<code>`watchEndpointSlices()`</code>	<code>`discovery.v1.EndpointSlice`</code>	<code>`onEndpointSliceUpsert`, `onEndpointSliceDelete`</code>

Each uses exponential backoff on failure and automatically reconnects.

Reconciler Logic

Service events (onServiceUpsert):

1. Parse ClusterIP, port, protocol
2. Compute deterministic serviceID (FNV-1a hash of UID + port)
3. Skip if already created (tracked in createdServices map)
4. Call CNCClient.API().CreateLoadBalancerService(serviceID, lbInfo)

EndpointSlice events (onEndpointSliceUpsert):

1. Build desired backend set from EndpointSlice addresses
2. Diff against tracked state (svcBackends map) -> compute added/removed
3. For added: compute deterministic backendID (FNV-1a hash of IP:Port)
4. Call CreateLoadBalancerBackends(addedBackends) ? registers in global table

- 5. Call UpdateLoadBalancerServiceBackends(serviceID, lbInfo, allNew, allOld) ? swap/link
- 6. For removed: call DeleteLoadBalancerBackends(af, ids) ? removes from global table
- 7. Update local state

State Tracking (In-Memory)

Map	Key	Value	Purpose
-----	-----	-----	-----
`svcBackends`	`"namespace/endpointslice"`	`[]BackendInfo`	Track backends per EndpointSlice
`createdServices`	`serviceID (uint16)`	`bool`	Prevent duplicate service creation

4. cncshim Library (vendor/github.com/princepereira/cncshim)

Role: Go bindings for the CNC API DLL. This is where actual eBPF map writes happen.

Architecture:

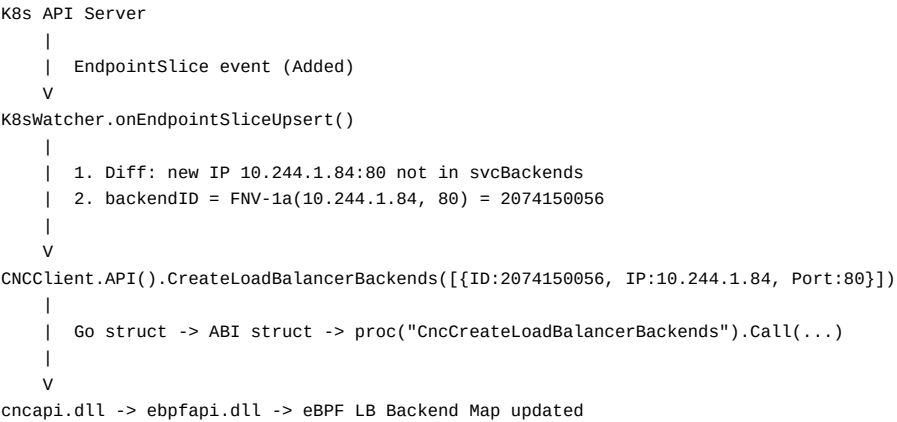
- pkg/cncapi/cnciface.go ? Defines CNCApi interface
- pkg/cncapi/client.go ? Implementation using syscall.LazyDLL
- pkg/cncapi/abi_types.go ? C-compatible struct definitions
- pkg/cncapi/abi_convert.go ? Go ? ABI conversion functions

DLL calling convention:

Go struct (BackendInfo) -> ABI struct (abiBackendInfo) -> syscall.Call -> cncapi.dll -> eBPF map

5. CNC API Call Flow (Complete Example)

When a new pod endpoint is discovered:



```

|
V
CNCCClient.API().UpdateLoadBalancerServiceBackends(serviceID, lbInfo, newSet, oldSet)
|
| Links backend to service (swap semantics: oldSet -> newSet)
|
V
cncapi.dll -> ebpfapi.dll -> eBPF LB Frontend Map updated (VIP -> backends)

```

Key Design Decisions

Decision	Rationale
Deterministic BackendID (FNV-1a of IP:Port)	CNC identifies backends by ID. Must survive agent restarts.
Deterministic ServiceID (FNV-1a of UID + Port)	Same service must get same ID across events.
Skip duplicate CreateLoadBalancerService	CNC re-creates on duplicate call, wiping backend associations.
Swap semantics for UpdateLoadBalancerServiceBackends	API requires full old+new sets, not incremental delta.
"Already exists" tolerance on CreateLoadBalancerBackends	Backend may exist from prior run; that's OK since ID is deterministic.

File Map

```

daemon/
  cmd/
    windows_stubs.go    ? Agent cell definition, NewAgentCmd, Execute
    main.go             ? Entry point: calls cmd.Execute(cmd.NewAgentCmd(hiveFn))

pkg/datapath/win/
  client.go             ? CNCCClient: lifecycle, retry, cncshim connection
  k8s_watcher.go        ? K8sWatcher: watch loops, reconciler, CNC API calls
  convert.go            ? Helper conversions
  README.md             ? Build instructions + prerequisites

vendor/github.com/princepereira/cncshim/
  pkg/cncapi/
    cnciface.go         ? CNCApi interface definition
    client.go           ? DLL-backed implementation
    abi_types.go        ? C-compatible struct layouts
    abi_convert.go      ? Go ? ABI conversions
    types.go            ? Public Go types (BackendInfo, LoadBalancerInfo, etc.)

```

Which Component Uses cncshim?

pkg/datapath/win/k8s_watcher.go is the component that imports and uses cncshim (via the CNCCClient wrapper in client.go) to make CNC API calls that update Windows eBPF maps.

The import chain:

k8s_watcher.go -> CNCCClient.API() -> cncapi.CNCApi interface -> cncapi.dll -> eBPF maps

client.go imports github.com/princepereira/cncshim/pkg/cncapi directly and wraps it with lifecycle management. k8s_watcher.go receives the *CNCCClient via Hive dependency injection and calls client.API() to get the interface.